

Marathon Platform Design System & Technical Brief

Project: Anti-Stress Marathon Engine (newday platform implementation)

Target Stack: React + TypeScript + Tailwind CSS + shadcn/ui

Date: February 27, 2026

Based on: AI Studio Marathon Engine MV (bf5eb4e6 version)

Executive Summary

This document provides complete design specifications, component guidelines, and technical requirements for implementing the Marathon Platform interface. The platform enables creation and delivery of psychological marathons/courses through three core modules: Architect (course builder), Runner (bot simulator), and Settings (integrations).

Design Philosophy

Core Principles

- **Technical Dashboard Aesthetic:** Professional, data-driven interface for administrators creating marathon content
- **Clean Utility Design:** Minimal, distraction-free experience for end users consuming content through chat simulator
- **Modular Architecture:** Each screen operates independently with clear separation of concerns
- **Accessibility First:** WCAG 2.1 AA compliance, keyboard navigation, screen reader support
- **Mobile-Ready:** Responsive layouts, touch-friendly targets (minimum 44×44px)

Visual Mood

The interface should feel calm, professional, and supportive—matching the anti-stress mission of the marathons themselves. Avoid visual noise, aggressive colors, or cluttered layouts. Use whitespace generously.

Color System

Primary Palette

```
/* Main Brand Colors /  
--color-primary: #4F46E5; / Indigo 600 - primary actions, links /  
--color-primary-hover: #4338CA; / Indigo 700 - hover states /  
--color-primary-light: #818CF8; / Indigo 400 - accents */  
  
/* Neutral Grays (Technical Dashboard) /  
--color-bg-primary: #0F172A; / Slate 900 - main background /  
--color-bg-secondary: #1E293B; / Slate 800 - cards, panels /  
--color-bg-tertiary: #334155; / Slate 700 - hover surfaces */  
  
--color-text-primary: #F8FAFC; /* Slate 50 - primary text /  
--color-text-secondary: #CBD5E1; / Slate 300 - secondary text /  
--color-text-muted: #64748B; / Slate 500 - hints, placeholders */  
  
/* Light Mode Alternative (Clean Utility) /  
--color-bg-light: #FFFFFF; / White background /  
--color-bg-light-secondary: #F8FAFC; / Slate 50 - subtle surfaces /  
--color-text-dark: #0F172A; / Slate 900 - text on light /  
--color-text-dark-secondary: #475569; / Slate 600 - secondary on light  
*/
```

Semantic Colors

```
/* Status & Feedback /  
--color-success: #10B981; / Emerald 500 - completed, success */  
--color-success-bg: rgba(16, 185, 129, 0.1);  
  
--color-warning: #F59E0B; /* Amber 500 - warnings, pending */  
--color-warning-bg: rgba(245, 158, 11, 0.1);
```

```
--color-error: #EF4444; /* Red 500 - errors, destructive */
--color-error-bg: rgba(239, 68, 68, 0.1);
```

```
--color-info: #3B82F6; /* Blue 500 - informational */
--color-info-bg: rgba(59, 130, 246, 0.1);
```

Block Type Colors (Content Blocks)

```
/* Specific to content block types /
--color-video: #8B5CF6; /* Purple 500 - VIDEO blocks /
--color-article: #06B6D4; /* Cyan 500 - ARTICLE blocks /
--color-quiz: #F59E0B; /* Amber 500 - QUIZ blocks /
--color-practice: #10B981; /* Emerald 500 - PRACTICE blocks /
--color-checkin: #EC4899; /* Pink 500 - CHECKIN blocks */
```

Usage Guidelines

- **Admin/Architect screens:** Use dark mode (slate 900 background) for technical dashboard feel
- **Runner/Simulator screens:** Use light mode (white background) for clean chat interface
- **Accent sparingly:** Primary indigo only for CTAs, active states, important actions
- **Block colors:** Apply to icons, badges, and left borders to differentiate content types

Typography

Font Families

```
/* System Font Stack */
--font-sans: -apple-system, BlinkMacSystemFont, 'Segoe UI',
'Inter', 'Helvetica Neue', Arial, sans-serif;

--font-mono: 'JetBrains Mono', 'Fira Code', 'Consolas',
'Monaco', monospace;
```

Note: If using custom fonts, prefer Inter or Public Sans for clean, readable UI.

Type Scale

/ Headings /*

--text-4xl: 2.25rem; / 36px - Page titles /

--text-3xl: 1.875rem; / 30px - Section headers /

--text-2xl: 1.5rem; / 24px - Card headers /

--text-xl: 1.25rem; / 20px - Subsection titles /

*--text-lg: 1.125rem; / 18px - Emphasized text */*

/ Body /*

--text-base: 1rem; / 16px - Body text, buttons /

--text-sm: 0.875rem; / 14px - Captions, labels /

*--text-xs: 0.75rem; / 12px - Tiny labels, badges */*

/ Line Heights /*

--leading-tight: 1.25; / Headings /

--leading-normal: 1.5; / Body text /

*--leading-relaxed: 1.75; / Long-form content */*

Font Weights

--font-normal: 400;

--font-medium: 500; / UI elements, buttons /*

--font-semibold: 600; / Headings, emphasis /

*--font-bold: 700; / Rare, strong emphasis */*

Typography Rules

- Headings: semibold (600), tight line-height
- Body text: normal (400), 16px minimum size
- Labels/inputs: medium (500), 14px
- Buttons: medium (500), 16px
- Code snippets: mono font, 14px

Spacing & Layout

Spacing Scale

```
--space-1: 0.25rem; /* 4px /  
--space-2: 0.5rem; / 8px /  
--space-3: 0.75rem; / 12px /  
--space-4: 1rem; / 16px /  
--space-5: 1.25rem; / 20px /  
--space-6: 1.5rem; / 24px /  
--space-8: 2rem; / 32px /  
--space-10: 2.5rem; / 40px /  
--space-12: 3rem; / 48px /  
--space-16: 4rem; / 64px */
```

Layout Grid

- **Container max-width:** 1280px (xl breakpoint)
- **Sidebar width:** 280px (Architect mode navigation)
- **Content padding:** 24px (mobile), 32px (tablet), 48px (desktop)
- **Card spacing:** 16px gap between cards
- **Section spacing:** 48px between major sections

Responsive Breakpoints

```
--breakpoint-sm: 640px;  
--breakpoint-md: 768px;  
--breakpoint-lg: 1024px;  
--breakpoint-xl: 1280px;
```

Border Radius & Shadows

Radius Scale

```
--radius-sm: 0.375rem; /* 6px - small elements /  
--radius-md: 0.5rem; / 8px - cards, inputs /  
--radius-lg: 0.75rem; / 12px - large cards /  
--radius-xl: 1rem; / 16px - modals /  
--radius-2xl: 1.5rem; / 24px - hero elements /  
--radius-full: 9999px; / Pills, avatars */
```

Key decision from AI Studio spec: Use **large radius values** (12px+) for modern, friendly feel.

Shadow System

```
/* Elevation Shadows */
--shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);
--shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1),
0 2px 4px -1px rgba(0, 0, 0, 0.06);
--shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1),
0 4px 6px -2px rgba(0, 0, 0, 0.05);
--shadow-xl: 0 20px 25px -5px rgba(0, 0, 0, 0.1),
0 10px 10px -5px rgba(0, 0, 0, 0.04);

/* Focus Ring */
--shadow-focus: 0 0 0 3px rgba(79, 70, 229, 0.3);
```

- Cards: shadow-md
- Modals/Dialogs: shadow-xl
- Dropdowns: shadow-lg
- Focus states: shadow-focus + outline-none

Component Library

1. Buttons

Primary Button

// Usage: Main CTAs, save actions, submissions

Create Course

Secondary Button

// Usage: Cancel, back, alternative actions

Cancel

Ghost Button

// Usage: Tertiary actions, icon buttons

View Details

Icon Button

// Usage: Toolbars, compact actions

2. Cards

Course Card (Architect View)

7-Day Anti-Stress Marathon

Active

Complete psychological program for stress management

7 days • 23 blocks • 156 enrollments

Day Card (Course Builder)

Day 1: Foundation

Introduction to stress management basics

2 Videos 1 Article

Content Block Card

{/* Left indicator by type */}

Video

Understanding Stress Triggers

8-minute guided video explanation

3. Forms & Inputs

Text Input

Course Title

Textarea

Select Dropdown

Select block type... Video Article Quiz

Toggle Switch

☐

Skippable block

4. Navigation

Top Navigation Bar

Marathon Platform

[Architect Runner Settings](#)

Sidebar Navigation (Architect)

My Courses Templates Analytics

5. Modals & Dialogs

Modal Overlay & Container

Create New Course

{/* Modal content */}

Cancel Create

6. Status Badges

{/* Success/Active */}

Active

{/* Warning/Draft */}

Draft

{/* Error/Inactive */}

Inactive

{/* Info */}

In Progress

7. Chat Simulator (Runner)

Chat Container (Light Mode)

{/* Header */}

Marathon Bot

Active now

{/* Messages */}

{/* Bot message */}

Welcome to Day 1! Ready to start?

{/* User message */}

Yes, let's begin!

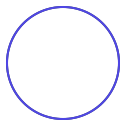
{/* Input */}

8. Progress Indicators

Linear Progress Bar

Day 3 of 7 43%

Circular Progress (Day Completion)



75%

Screen Layouts

1. Architect: Course List

Layout Structure:

Top Nav Bar	
Side bar	Main Content Area
	- Header with "Create" button
	- Search/Filter bar
	- Grid of Course Cards

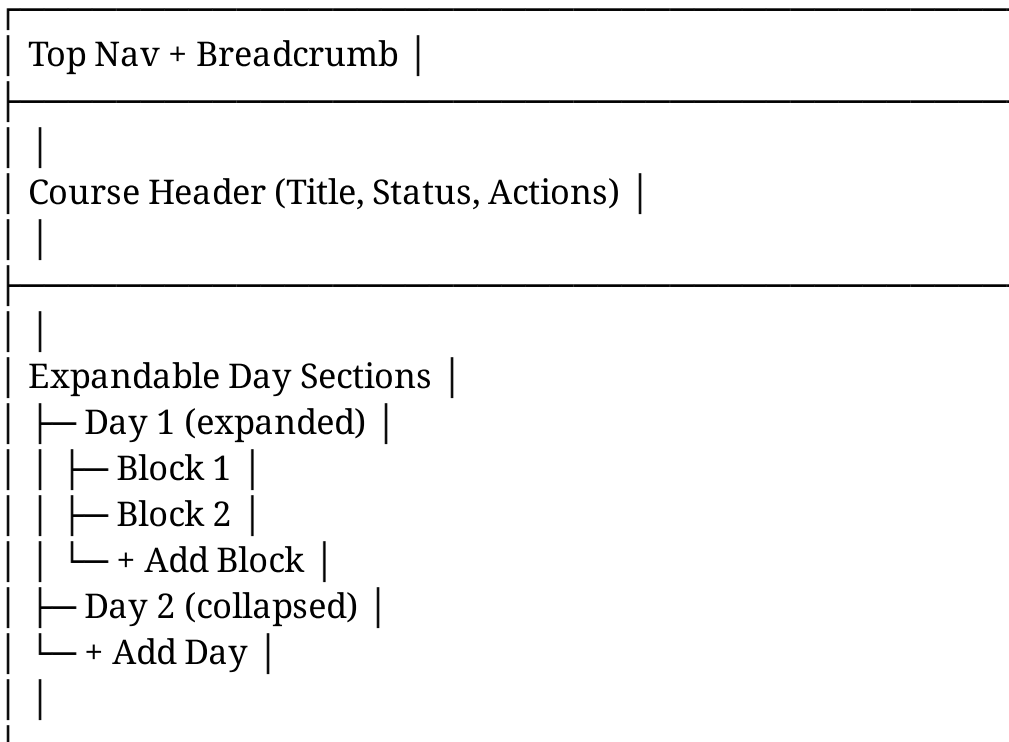


Key Elements:

- Top nav: Fixed, 64px height, dark slate-900
- Sidebar: 280px width, slate-900, sticky navigation
- Main content: Max 1280px width, 48px padding
- Course cards: 3-column grid on desktop, 2 on tablet, 1 on mobile
- Gap between cards: 16px

2. Architect: Course Builder

Layout Structure:



Key Features:

- Accordion-style day sections
- Drag-and-drop reordering (visual affordance: handle icon)
- Inline editing: click to edit titles/descriptions
- "Add Block" button within each day
- Right sidebar (optional): Block properties panel

3. Runner: Bot Simulator

Layout Structure:

Chat Header (Avatar, Name, Status)
Message Thread
- Bot messages (left-aligned)
- User messages (right-aligned)
- Block content (video, quiz, etc)
Input Bar (Message + Send button)

Key Features:

- Light mode: white background, slate-50 input area
- Messages with timestamps (subtle, slate-400)
- Block content rendered as cards within chat
- Auto-scroll to latest message
- Typing indicator for bot responses

4. Settings: Integration Config

Layout Structure:

Top Nav
Settings Sections (tabs or accordion)
└─ Webhook Configuration
└─ Telegram Integration
└─ Database Sync
└─ API Keys

Key Elements:

- Tabbed navigation or left sidebar for sections
- Form fields with clear labels and help text
- "Test Connection" buttons for integrations
- Success/error toast notifications
- Save button in sticky footer or top-right

Data Structure Reference

Course Schema

```
interface Course {  
  id: string;  
  title: string;  
  description: string;  
  status: 'draft' | 'active' | 'archived';  
  durationDays: number;  
  createdAt: Date;  
  updatedAt: Date;  
  days: Day[];  
}  
  
interface Day {  
  id: string;  
  courseId: string;  
  dayNumber: number;  
  title: string;  
  description: string;  
  blocks: Block[];  
}  
  
interface Block {  
  id: string;  
  dayId: string;  
  type: 'VIDEO' | 'ARTICLE' | 'QUIZ' | 'PRACTICE' | 'CHECKIN';  
  order: number;  
  title: string;  
  content: BlockContent;  
  settings: {  
    skippable: boolean;  
    required: boolean;
```

```
timeEstimate?: number; // minutes
};
}

interface BlockContent {
// Type-specific content
videoUrl?: string;
articleHtml?: string;
quizQuestions?: QuizQuestion[];
practiceInstructions?: string;
checkInPrompt?: string;
}

interface Enrollment {
id: string;
userId: string;
courseId: string;
startedAt: Date;
currentDay: number;
progress: Progress[];
status: 'active' | 'completed' | 'paused';
}

interface Progress {
blockId: string;
completedAt: Date | null;
response?: any; // User response for quiz/checkin
}
```

Block Type Specifications

Type	Icon	Color	Content Fields
VIDEO	IconVideo	Purple (8B5CF6)	videoUrl, duration
ARTICLE	IconDocument	Cyan (06B6D4)	articleHtml, readTime
QUIZ	IconQuestionMark	Amber (F59E0B)	questions[], passingScore
PRACTICE	IconPencil	Emerald (10B981)	instructions, template
CHECKIN	IconHeart	Pink (EC4899)	prompt, responseType

Table 1: Content block type specifications

Implementation Checklist

Phase 1: Foundation (Week 1)

- ☐ Set up Tailwind CSS with custom color variables
- ☐ Create base component library (Button, Card, Input, Modal)
- ☐ Implement dark/light theme toggle
- ☐ Set up routing (Architect, Runner, Settings)
- ☐ Design responsive layout shell

Phase 2: Architect Module (Week 2-3)

- ☐ Course list view with grid layout
- ☐ Course creation modal
- ☐ Course builder with day/block structure
- ☐ Drag-and-drop reordering
- ☐ Block type editor forms (VIDEO, ARTICLE, QUIZ, etc)
- ☐ Save/publish functionality

Phase 3: Runner Module (Week 4)

- ☐ Chat interface layout (light mode)
- ☐ Message rendering (bot/user)
- ☐ Block content display within chat
- ☐ Progress tracking UI

- ☐ User response collection (quiz answers, checkins)

Phase 4: Settings & Integration (Week 5)

- ☐ Webhook configuration UI
- ☐ Telegram bot integration form
- ☐ Database sync settings
- ☐ Test connection buttons
- ☐ API key management

Phase 5: Polish & Deploy (Week 6)

- ☐ Accessibility audit (WCAG 2.1 AA)
- ☐ Mobile responsiveness testing
- ☐ Loading states and error handling
- ☐ Toast notifications system
- ☐ Deploy to production (match newday deployment setup)

shadcn/ui Component Mapping

Use these shadcn components as base, then customize with the color system above:

- Button → Customize with indigo-600 primary, slate-700 secondary
- Card → Use slate-800 background, slate-700 border
- Input → Dark slate-700 background, slate-600 border
- Select → Match input styling
- Dialog → Modal system with slate-800 background
- Tabs → For Settings navigation
- Accordion → For expandable day sections
- Badge → Status indicators with semantic colors
- Progress → Linear progress bars
- Switch → Toggle components

Installation command:

```
npx shadcn-ui@latest init
```

```
npx shadcn-ui@latest add button card input select dialog tabs  
accordion badge progress switch
```

Accessibility Requirements

- **Keyboard Navigation:** All interactive elements accessible via Tab, Enter, Escape
- **Focus Indicators:** Visible focus ring (3px indigo-300 shadow)
- **Color Contrast:** Minimum 4.5:1 for normal text, 3:1 for large text
- **Screen Readers:** Proper ARIA labels, landmarks, live regions
- **Touch Targets:** Minimum 44×44px on mobile
- **Error Messages:** Clear, associated with form fields via aria-describedby

Animation Guidelines

Use subtle, purposeful animations:

```
/* Standard transition */
```

```
transition: all 200ms cubic-bezier(0.4, 0, 0.2, 1);
```

```
/* Hover states */
```

```
transition: background-color 200ms, box-shadow 200ms;
```

```
/* Modal enter */
```

```
animation: fadeIn 200ms ease-out;
```

```
/* Page transitions */
```

```
animation: slideIn 300ms ease-out;
```

Avoid:

- Excessive motion (respect prefers-reduced-motion)
- Long animation durations (>400ms)
- Animations on every interaction (reserve for important feedback)

Code Style Guidelines

Tailwind Class Ordering

Follow this order for consistency:

1. Layout (display, position, flex/grid)
2. Sizing (width, height, padding, margin)
3. Typography (font, text properties)
4. Visual (background, border, shadow)
5. Interactive (hover, focus, transition)

Example:

```
className="
flex items-center justify-between
w-full px-4 py-3
text-sm font-medium text-white
bg-indigo-600 rounded-lg shadow-md
hover:bg-indigo-700 focus:ring-2 focus:ring-indigo-300
transition-all duration-200
"
```

Component Structure

```
// CourseCard.tsx
interface CourseCardProps {
  course: Course;
  onEdit: (id: string) => void;
  onDelete: (id: string) => void;
}

export function CourseCard({ course, onEdit, onDelete }:
CourseCardProps) {
  return (

    {/* Component content */}

  );
}
```

File Organization

```
src/
├── components/
│   ├── ui/ # shadcn components + custom base components
│   │   ├── button.tsx
│   │   ├── card.tsx
│   │   └── input.tsx
│   ├── architect/ # Architect module components
│   │   ├── CourseCard.tsx
│   │   ├── DaySection.tsx
│   │   └── BlockEditor.tsx
│   ├── runner/ # Runner module components
│   │   ├── ChatMessage.tsx
│   │   └── BlockContent.tsx
│   └── layout/ # Layout components
│       ├── Navbar.tsx
│       └── Sidebar.tsx
├── pages/ # Route pages
│   ├── ArchitectPage.tsx
│   ├── RunnerPage.tsx
│   └── SettingsPage.tsx
├── lib/ # Utilities, API clients
├── types/ # TypeScript interfaces
└── styles/ # Global CSS, Tailwind config
```

Integration Notes

Connecting to newday Backend

The newday project has backend infrastructure. Connect this UI via:

```
// lib/api.ts
const API_BASE_URL = import.meta.env.VITE_API_URL;

export async function fetchCourses(): Promise<Course[]> {
  const response = await fetch(`${API_BASE_URL}/api/courses`);
  return response.json();
}
```

```
export async function saveCourse(course: Course): Promise<Course>
{
  const response = await fetch(`${API_BASE_URL}/api/courses`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(course),
  });
  return response.json();
}
```

Telegram Bot Integration

From AI Studio spec: Use webhook pattern for n8n integration.

Settings page should allow configuring:

- Webhook URL (n8n endpoint)
- Bot token (Telegram)
- Event triggers (enrollment, completion, daily reminder)

localStorage (MVP Mode)

For initial development without backend:

```
// lib/storage.ts
export function saveCourse(course: Course) {
  const courses = getCourses();
  courses.push(course);
  localStorage.setItem('courses', JSON.stringify(courses));
}

export function getCourses(): Course[] {
  const data = localStorage.getItem('courses');
  return data ? JSON.parse(data) : [];
}
```

Testing Recommendations

- **Component Tests:** Jest + React Testing Library for UI components

- **E2E Tests:** Playwright for critical user flows (create course, run simulator)
- **Visual Regression:** Chromatic or Percy for design consistency
- **Accessibility:** axe-core automated testing + manual screen reader testing

Deployment Configuration

Match the newday deployment approach:

- Build with Vite: `npm run build`
- Output to `dist/`
- Serve via nginx or integrated with existing newday infrastructure
- Environment variables for API endpoint, webhook URLs
- SSL via Traefik (as documented in newday `SERVER_DEPLOYMENT`)

Design Tokens Export

For design tools integration (Figma, etc):

```
{
  "colors": {
    "primary": "#4F46E5",
    "bg-dark": "#0F172A",
    "text-light": "#F8FAFC"
  },
  "spacing": {
    "unit": "4px",
    "scale": [4, 8, 12, 16, 20, 24, 32, 40, 48, 64]
  },
  "typography": {
    "fontFamily": "Inter, system-ui, sans-serif",
    "scale": {
      "xs": "12px",
      "sm": "14px",
      "base": "16px",
      "lg": "18px",
      "xl": "20px",
    }
  }
}
```

```
"2xl": "24px",  
"3xl": "30px",  
"4xl": "36px"  
}  
,  
"borderRadius": {  
  "sm": "6px",  
  "md": "8px",  
  "lg": "12px",  
  "xl": "16px",  
  "2xl": "24px"  
}  
}
```

Quick Start Commands

1. Clone newday repo (or marathon-craft if preferred)

```
git clone <repo-url>
```

2. Install dependencies

```
npm install
```

3. Install shadcn components

```
npx shadcn-ui@latest init
```

```
npx shadcn-ui@latest add button card input select dialog tabs  
accordion badge progress switch
```

4. Start development server

`npm run dev`

5. Build for production

`npm run build`

Support & Questions

When implementing this design system in Cursor:

1. Reference this document for all color, spacing, typography decisions
2. Use the component examples as starting templates
3. Match the layout structures described for each screen
4. Follow the data structure schema for state management
5. Consult the checklist to prioritize features

For questions about specific design decisions or implementation approaches, refer back to the AI Studio Marathon Engine spec (bf5eb4e6 version) and the newday project documentation.

Document Version: 1.0

Last Updated: February 27, 2026

Author: Marathon Platform Team

Marathon Platform Design System & Technical Brief

Project: Anti-Stress Marathon Engine (newday platform implementation)

Target Stack: React + TypeScript + Tailwind CSS + shadcn/ui

Date: February 27, 2026

Based on: AI Studio Marathon Engine MV (bf5eb4e6 version)

Executive Summary

This document provides complete design specifications, component guidelines, and technical requirements for implementing the Marathon Platform interface. The platform enables creation and delivery of psychological marathons/courses through three core modules: Architect (course builder), Runner (bot simulator), and Settings (integrations).

Design Philosophy

Core Principles

- **Technical Dashboard Aesthetic:** Professional, data-driven interface for administrators creating marathon content
- **Clean Utility Design:** Minimal, distraction-free experience for end users consuming content through chat simulator
- **Modular Architecture:** Each screen operates independently with clear separation of concerns
- **Accessibility First:** WCAG 2.1 AA compliance, keyboard navigation, screen reader support
- **Mobile-Ready:** Responsive layouts, touch-friendly targets (minimum 44×44px)

Visual Mood

The interface should feel calm, professional, and supportive—matching the anti-stress mission of the marathons themselves. Avoid visual noise, aggressive colors, or cluttered layouts. Use whitespace generously.

Color System

Primary Palette

```
/* Main Brand Colors /  
--color-primary: #4F46E5; / Indigo 600 - primary actions, links /  
--color-primary-hover: #4338CA; / Indigo 700 - hover states /  
--color-primary-light: #818CF8; / Indigo 400 - accents */
```

```
/* Neutral Grays (Technical Dashboard) /
--color-bg-primary: #0F172A; / Slate 900 - main background /
--color-bg-secondary: #1E293B; / Slate 800 - cards, panels /
--color-bg-tertiary: #334155; / Slate 700 - hover surfaces */

--color-text-primary: #F8FAFC; /* Slate 50 - primary text /
--color-text-secondary: #CBD5E1; / Slate 300 - secondary text /
--color-text-muted: #64748B; / Slate 500 - hints, placeholders */

/* Light Mode Alternative (Clean Utility) /
--color-bg-light: #FFFFFF; / White background /
--color-bg-light-secondary: #F8FAFC; / Slate 50 - subtle surfaces /
--color-text-dark: #0F172A; / Slate 900 - text on light /
--color-text-dark-secondary: #475569; / Slate 600 - secondary on light
*/
```

Semantic Colors

```
/* Status & Feedback /
--color-success: #10B981; / Emerald 500 - completed, success */
--color-success-bg: rgba(16, 185, 129, 0.1);

--color-warning: #F59E0B; /* Amber 500 - warnings, pending */
--color-warning-bg: rgba(245, 158, 11, 0.1);

--color-error: #EF4444; /* Red 500 - errors, destructive */
--color-error-bg: rgba(239, 68, 68, 0.1);

--color-info: #3B82F6; /* Blue 500 - informational */
--color-info-bg: rgba(59, 130, 246, 0.1);
```

Block Type Colors (Content Blocks)

```
/* Specific to content block types /
--color-video: #8B5CF6; / Purple 500 - VIDEO blocks /
--color-article: #06B6D4; / Cyan 500 - ARTICLE blocks /
--color-quiz: #F59E0B; / Amber 500 - QUIZ blocks /
--color-practice: #10B981; / Emerald 500 - PRACTICE blocks /
--color-checkin: #EC4899; / Pink 500 - CHECKIN blocks */
```

Usage Guidelines

- **Admin/Architect screens:** Use dark mode (slate 900 background) for technical dashboard feel
- **Runner/Simulator screens:** Use light mode (white background) for clean chat interface
- **Accent sparingly:** Primary indigo only for CTAs, active states, important actions
- **Block colors:** Apply to icons, badges, and left borders to differentiate content types

Typography

Font Families

/ System Font Stack */*

--font-sans: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Inter', 'Helvetica Neue', Arial, sans-serif;

--font-mono: 'JetBrains Mono', 'Fira Code', 'Consolas', 'Monaco', monospace;

Note: If using custom fonts, prefer Inter or Public Sans for clean, readable UI.

Type Scale

/ Headings /*

--text-4xl: 2.25rem; / 36px - Page titles /

--text-3xl: 1.875rem; / 30px - Section headers /

--text-2xl: 1.5rem; / 24px - Card headers /

--text-xl: 1.25rem; / 20px - Subsection titles /

*--text-lg: 1.125rem; / 18px - Emphasized text */*

/ Body /*

--text-base: 1rem; / 16px - Body text, buttons /

--text-sm: 0.875rem; / 14px - Captions, labels /

*--text-xs: 0.75rem; / 12px - Tiny labels, badges */*

/ Line Heights /*

--leading-tight: 1.25; / Headings /

--leading-normal: 1.5; / Body text /
--leading-relaxed: 1.75; / Long-form content */

Font Weights

--font-normal: 400;
--font-medium: 500; /* UI elements, buttons /
--font-semibold: 600; / Headings, emphasis /
--font-bold: 700; / Rare, strong emphasis */

Typography Rules

- Headings: semibold (600), tight line-height
- Body text: normal (400), 16px minimum size
- Labels/inputs: medium (500), 14px
- Buttons: medium (500), 16px
- Code snippets: mono font, 14px

Spacing & Layout

Spacing Scale

--space-1: 0.25rem; /* 4px /
--space-2: 0.5rem; / 8px /
--space-3: 0.75rem; / 12px /
--space-4: 1rem; / 16px /
--space-5: 1.25rem; / 20px /
--space-6: 1.5rem; / 24px /
--space-8: 2rem; / 32px /
--space-10: 2.5rem; / 40px /
--space-12: 3rem; / 48px /
--space-16: 4rem; / 64px */

Layout Grid

- **Container max-width:** 1280px (xl breakpoint)
- **Sidebar width:** 280px (Architect mode navigation)
- **Content padding:** 24px (mobile), 32px (tablet), 48px (desktop)
- **Card spacing:** 16px gap between cards
- **Section spacing:** 48px between major sections

Responsive Breakpoints

```
--breakpoint-sm: 640px;  
--breakpoint-md: 768px;  
--breakpoint-lg: 1024px;  
--breakpoint-xl: 1280px;
```

Border Radius & Shadows

Radius Scale

```
--radius-sm: 0.375rem; /* 6px - small elements /  
--radius-md: 0.5rem; / 8px - cards, inputs /  
--radius-lg: 0.75rem; / 12px - large cards /  
--radius-xl: 1rem; / 16px - modals /  
--radius-2xl: 1.5rem; / 24px - hero elements /  
--radius-full: 9999px; / Pills, avatars */
```

Key decision from AI Studio spec: Use **large radius values** (12px+) for modern, friendly feel.

Shadow System

```
/* Elevation Shadows */  
--shadow-sm: 0 1px 2px 0 rgba(0, 0, 0, 0.05);  
--shadow-md: 0 4px 6px -1px rgba(0, 0, 0, 0.1),  
0 2px 4px -1px rgba(0, 0, 0, 0.06);  
--shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1),  
0 4px 6px -2px rgba(0, 0, 0, 0.05);  
--shadow-xl: 0 20px 25px -5px rgba(0, 0, 0, 0.1),  
0 10px 10px -5px rgba(0, 0, 0, 0.04);  
  
/* Focus Ring */  
--shadow-focus: 0 0 0 3px rgba(79, 70, 229, 0.3);
```

- Cards: shadow-md
- Modals/Dialogs: shadow-xl
- Dropdowns: shadow-lg
- Focus states: shadow-focus + outline-none

Component Library

1. Buttons

Primary Button

// Usage: Main CTAs, save actions, submissions

Create Course

Secondary Button

// Usage: Cancel, back, alternative actions

Cancel

Ghost Button

// Usage: Tertiary actions, icon buttons

View Details

Icon Button

// Usage: Toolbars, compact actions

2. Cards

Course Card (Architect View)

7-Day Anti-Stress Marathon

Active

Complete psychological program for stress management

7 days • 23 blocks • 156 enrollments

Day Card (Course Builder)

Day 1: Foundation

Introduction to stress management basics

2 Videos 1 Article

Content Block Card

{/* Left indicator by type */}

Video

Understanding Stress Triggers

8-minute guided video explanation

3. Forms & Inputs

Text Input

Course Title

Textarea

Select Dropdown

Select block type... Video Article Quiz

Toggle Switch

☐

Skippable block

4. Navigation

Top Navigation Bar

Marathon Platform

[Architect Runner Settings](#)

Sidebar Navigation (Architect)

My Courses Templates Analytics

5. Modals & Dialogs

Modal Overlay & Container

Create New Course

{/* Modal content */}

Cancel Create

6. Status Badges

{/* Success/Active */}

Active

{/* Warning/Draft */}

Draft

{/* Error/Inactive */}

Inactive

{/* Info */}

In Progress

7. Chat Simulator (Runner)

Chat Container (Light Mode)

{/* Header */}

Marathon Bot

Active now

{/* Messages */}

{/* Bot message */}

Welcome to Day 1! Ready to start?

{/* User message */}

Yes, let's begin!

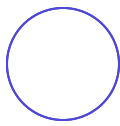
{/* Input */}

8. Progress Indicators

Linear Progress Bar

Day 3 of 7 43%

Circular Progress (Day Completion)



75%

Screen Layouts

1. Architect: Course List

Layout Structure:

Top Nav Bar	
Side Main Content Area	
bar - Header with "Create" button	
- Search/Filter bar	
- Grid of Course Cards	

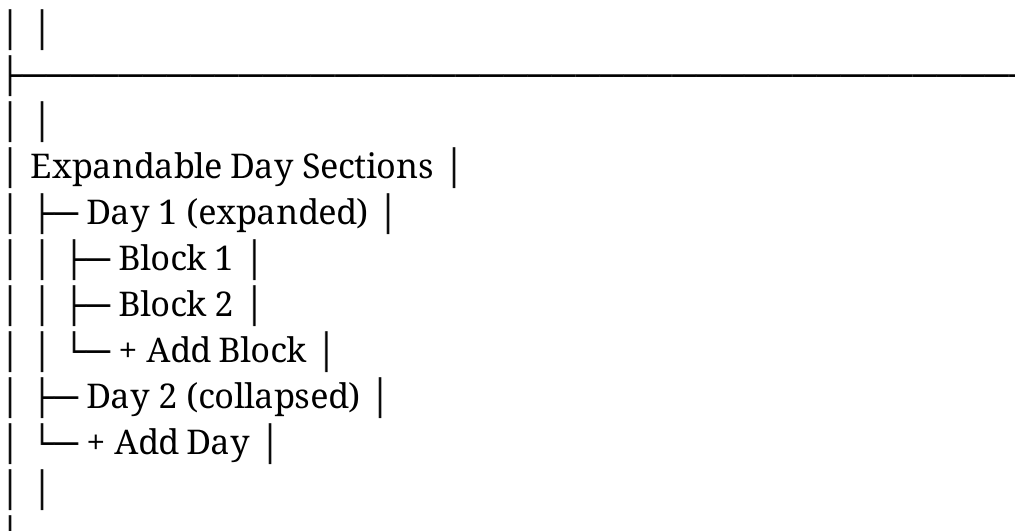
Key Elements:

- Top nav: Fixed, 64px height, dark slate-900
- Sidebar: 280px width, slate-900, sticky navigation
- Main content: Max 1280px width, 48px padding
- Course cards: 3-column grid on desktop, 2 on tablet, 1 on mobile
- Gap between cards: 16px

2. Architect: Course Builder

Layout Structure:

Top Nav + Breadcrumb	
Course Header (Title, Status, Actions)	

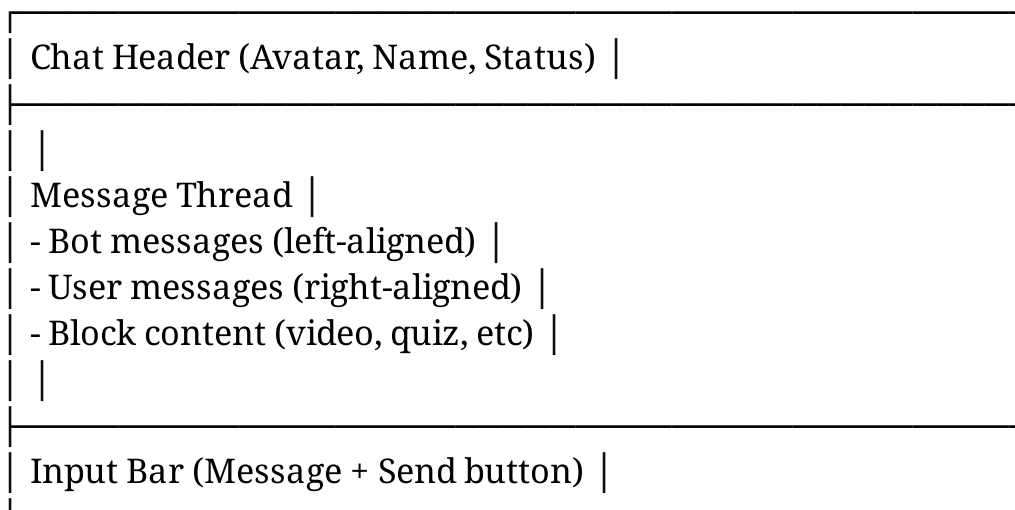


Key Features:

- Accordion-style day sections
- Drag-and-drop reordering (visual affordance: handle icon)
- Inline editing: click to edit titles/descriptions
- "Add Block" button within each day
- Right sidebar (optional): Block properties panel

3. Runner: Bot Simulator

Layout Structure:



Key Features:

- Light mode: white background, slate-50 input area
- Messages with timestamps (subtle, slate-400)

- Block content rendered as cards within chat
- Auto-scroll to latest message
- Typing indicator for bot responses

4. Settings: Integration Config

Layout Structure:

Top Nav
Settings Sections (tabs or accordion)
└─ Webhook Configuration
└─ Telegram Integration
└─ Database Sync
└─ API Keys

Key Elements:

- Tabbed navigation or left sidebar for sections
- Form fields with clear labels and help text
- "Test Connection" buttons for integrations
- Success/error toast notifications
- Save button in sticky footer or top-right

Data Structure Reference

Course Schema

```
interface Course {  
  id: string;  
  title: string;  
  description: string;  
  status: 'draft' | 'active' | 'archived';  
  durationDays: number;  
  createdAt: Date;  
  updatedAt: Date;
```

```
days: Day[];  
}
```

```
interface Day {  
  id: string;  
  courseId: string;  
  dayNumber: number;  
  title: string;  
  description: string;  
  blocks: Block[];  
}
```

```
interface Block {  
  id: string;  
  dayId: string;  
  type: 'VIDEO' | 'ARTICLE' | 'QUIZ' | 'PRACTICE' | 'CHECKIN';  
  order: number;  
  title: string;  
  content: BlockContent;  
  settings: {  
    skippable: boolean;  
    required: boolean;  
    timeEstimate?: number; // minutes  
  };  
}
```

```
interface BlockContent {  
  // Type-specific content  
  videoUrl?: string;  
  articleHtml?: string;  
  quizQuestions?: QuizQuestion[];  
  practiceInstructions?: string;  
  checkInPrompt?: string;  
}
```

```
interface Enrollment {  
  id: string;  
  userId: string;  
  courseId: string;  
  startedAt: Date;
```

```
currentDay: number;
progress: Progress[];
status: 'active' | 'completed' | 'paused';
}

interface Progress {
  blockId: string;
  completedAt: Date | null;
  response?: any; // User response for quiz/checkin
}
```

Block Type Specifications

Type	Icon	Color	Content Fields
VIDEO	IconVideo	Purple (8B5CF6)	videoUrl, duration
ARTICLE	IconDocument	Cyan (06B6D4)	articleHtml, readTime
QUIZ	IconQuestionMark	Amber (F59E0B)	questions[], passingScore
PRACTICE	IconPencil	Emerald (10B981)	instructions, template
CHECKIN	IconHeart	Pink (EC4899)	prompt, responseType

Table 2: Content block type specifications

Implementation Checklist

Phase 1: Foundation (Week 1)

- ☐ Set up Tailwind CSS with custom color variables
- ☐ Create base component library (Button, Card, Input, Modal)
- ☐ Implement dark/light theme toggle
- ☐ Set up routing (Architect, Runner, Settings)
- ☐ Design responsive layout shell

Phase 2: Architect Module (Week 2-3)

- ☐ Course list view with grid layout
- ☐ Course creation modal
- ☐ Course builder with day/block structure
- ☐ Drag-and-drop reordering
- ☐ Block type editor forms (VIDEO, ARTICLE, QUIZ, etc)
- ☐ Save/publish functionality

Phase 3: Runner Module (Week 4)

- ☐ Chat interface layout (light mode)
- ☐ Message rendering (bot/user)
- ☐ Block content display within chat
- ☐ Progress tracking UI
- ☐ User response collection (quiz answers, checkins)

Phase 4: Settings & Integration (Week 5)

- ☐ Webhook configuration UI
- ☐ Telegram bot integration form
- ☐ Database sync settings
- ☐ Test connection buttons
- ☐ API key management

Phase 5: Polish & Deploy (Week 6)

- ☐ Accessibility audit (WCAG 2.1 AA)
- ☐ Mobile responsiveness testing
- ☐ Loading states and error handling
- ☐ Toast notifications system
- ☐ Deploy to production (match newday deployment setup)

shadcn/ui Component Mapping

Use these shadcn components as base, then customize with the color system above:

- Button → Customize with indigo-600 primary, slate-700 secondary
- Card → Use slate-800 background, slate-700 border

- **Input** → Dark slate-700 background, slate-600 border
- **Select** → Match input styling
- **Dialog** → Modal system with slate-800 background
- **Tabs** → For Settings navigation
- **Accordion** → For expandable day sections
- **Badge** → Status indicators with semantic colors
- **Progress** → Linear progress bars
- **Switch** → Toggle components

Installation command:

`npx shadcn-ui@latest init`

`npx shadcn-ui@latest add button card input select dialog tabs`

`accordion badge progress switch`

Accessibility Requirements

- **Keyboard Navigation:** All interactive elements accessible via Tab, Enter, Escape
- **Focus Indicators:** Visible focus ring (3px indigo-300 shadow)
- **Color Contrast:** Minimum 4.5:1 for normal text, 3:1 for large text
- **Screen Readers:** Proper ARIA labels, landmarks, live regions
- **Touch Targets:** Minimum 44×44px on mobile
- **Error Messages:** Clear, associated with form fields via aria-describedby

Animation Guidelines

Use subtle, purposeful animations:

```
/* Standard transition */
```

```
transition: all 200ms cubic-bezier(0.4, 0, 0.2, 1);
```

```
/* Hover states */
```

```
transition: background-color 200ms, box-shadow 200ms;
```

```
/* Modal enter */
```

```
animation: fadeIn 200ms ease-out;
```

```
/* Page transitions */  
animation: slideIn 300ms ease-out;
```

Avoid:

- Excessive motion (respect prefers-reduced-motion)
- Long animation durations (>400ms)
- Animations on every interaction (reserve for important feedback)

Code Style Guidelines

Tailwind Class Ordering

Follow this order for consistency:

1. Layout (display, position, flex/grid)
2. Sizing (width, height, padding, margin)
3. Typography (font, text properties)
4. Visual (background, border, shadow)
5. Interactive (hover, focus, transition)

Example:

```
className="  
flex items-center justify-between  
w-full px-4 py-3  
text-sm font-medium text-white  
bg-indigo-600 rounded-lg shadow-md  
hover:bg-indigo-700 focus:ring-2 focus:ring-indigo-300  
transition-all duration-200  
"
```

Component Structure

```
// CourseCard.tsx  
interface CourseCardProps {  
  course: Course;  
  onEdit: (id: string) => void;  
  onDelete: (id: string) => void;  
}
```

```
export function CourseCard({ course, onEdit, onDelete }:  
CourseCardProps) {  
  return (  
  
    { /* Component content */ }  
  
  );  
}
```

File Organization

```
src/  
├── components/  
│   ├── ui/ # shadcn components + custom base components  
│   │   ├── button.tsx  
│   │   ├── card.tsx  
│   │   └── input.tsx  
│   ├── architect/ # Architect module components  
│   │   ├── CourseCard.tsx  
│   │   ├── DaySection.tsx  
│   │   └── BlockEditor.tsx  
│   ├── runner/ # Runner module components  
│   │   ├── ChatMessage.tsx  
│   │   └── BlockContent.tsx  
│   └── layout/ # Layout components  
│       ├── Navbar.tsx  
│       └── Sidebar.tsx  
├── pages/ # Route pages  
│   ├── ArchitectPage.tsx  
│   ├── RunnerPage.tsx  
│   └── SettingsPage.tsx  
├── lib/ # Utilities, API clients  
├── types/ # TypeScript interfaces  
└── styles/ # Global CSS, Tailwind config
```

Integration Notes

Connecting to newday Backend

The newday project has backend infrastructure. Connect this UI via:

```
// lib/api.ts
const API_BASE_URL = import.meta.env.VITE_API_URL;

export async function fetchCourses(): Promise<Course[]> {
  const response = await fetch(`${API_BASE_URL}/api/courses`);
  return response.json();
}

export async function saveCourse(course: Course): Promise<Course>
{
  const response = await fetch(`${API_BASE_URL}/api/courses`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(course),
  });
  return response.json();
}
```

Telegram Bot Integration

From AI Studio spec: Use webhook pattern for n8n integration.

Settings page should allow configuring:

- Webhook URL (n8n endpoint)
- Bot token (Telegram)
- Event triggers (enrollment, completion, daily reminder)

localStorage (MVP Mode)

For initial development without backend:

```
// lib/storage.ts
export function saveCourse(course: Course) {
  const courses = getCourses();
  courses.push(course);
}
```

```
localStorage.setItem('courses', JSON.stringify(courses));
}
```

```
export function getCourses(): Course[] {
  const data = localStorage.getItem('courses');
  return data ? JSON.parse(data) : [];
}
```

Testing Recommendations

- **Component Tests:** Jest + React Testing Library for UI components
- **E2E Tests:** Playwright for critical user flows (create course, run simulator)
- **Visual Regression:** Chromatic or Percy for design consistency
- **Accessibility:** axe-core automated testing + manual screen reader testing

Deployment Configuration

Match the newday deployment approach:

- Build with Vite: `npm run build`
- Output to `dist/`
- Serve via nginx or integrated with existing newday infrastructure
- Environment variables for API endpoint, webhook URLs
- SSL via Traefik (as documented in newday `SERVER_DEPLOYMENT`)

Design Tokens Export

For design tools integration (Figma, etc):

```
{
  "colors": {
    "primary": "#4F46E5",
    "bg-dark": "#0F172A",
    "text-light": "#F8FAFC"
  },
}
```

```
"spacing": {  
  "unit": "4px",  
  "scale": [4, 8, 12, 16, 20, 24, 32, 40, 48, 64]  
},  
"typography": {  
  "fontFamily": "Inter, system-ui, sans-serif",  
  "scale": {  
    "xs": "12px",  
    "sm": "14px",  
    "base": "16px",  
    "lg": "18px",  
    "xl": "20px",  
    "2xl": "24px",  
    "3xl": "30px",  
    "4xl": "36px"  
  }  
},  
"borderRadius": {  
  "sm": "6px",  
  "md": "8px",  
  "lg": "12px",  
  "xl": "16px",  
  "2xl": "24px"  
}  
}
```

Quick Start Commands

1. Clone newday repo (or marathon-craft if preferred)

```
git clone <repo-url>
```

2. Install dependencies

```
npm install
```

3. Install shadcn components

```
npx shadcn-ui@latest init  
npx shadcn-ui@latest add button card input select dialog tabs  
accordion badge progress switch
```

4. Start development server

```
npm run dev
```

5. Build for production

```
npm run build
```

Support & Questions

When implementing this design system in Cursor:

1. Reference this document for all color, spacing, typography decisions
2. Use the component examples as starting templates
3. Match the layout structures described for each screen
4. Follow the data structure schema for state management
5. Consult the checklist to prioritize features

For questions about specific design decisions or implementation approaches, refer back to the AI Studio Marathon Engine spec (bf5eb4e6 version) and the newday project documentation.

Document Version: 1.0

Last Updated: February 27, 2026

Author: Marathon Platform Team

